Dexy:
A Simple Stablecoin Design Based on an Algorithmic Central Bank

Alexander Chepurnoy
Ergo Platform
kushti@protonmail.ch

Amitabh Saxena
Ergo Platform

Luca D'Angelo
The Stable Order
ldgaetano@protonmail.com

Abstract

We consider a new algorithmic stablecoin called Dexy consisting of three components: a central bank, a secondary market in the form of a customized CP-AMM liquidity pool, and a trusted oracle. The price of the stablecoin in the liquidity pool is stabilized, relative to the oracle price, via central bank interventions. Users mint Dexy stablecoins from the bank using basecoins, the base cryptocurrency asset.

1 Introduction

Due to the inherent volatility of cryptocurrency prices, algorithmic stablecoins naturally arose as their extension to address this problem. The volatility is resolved by pegging the stablecoin to an asset whose price is considered to be stable over long periods of time, e.g. gold.

Having an asset with a stable value can be useful in many scenarios:

- Securing fundraising: a project can be sure that funds collected during fundraising will have stable value in the mid-to-long term.

We would would like to thank Ile for his inspiring forum posts, Bruno for discussions and the extract to future idea, and 4EYES Labs for the audit of the ErgoScript contracts.

- Doing business with predictable results: a shop can be sure that funds collected from sales will have roughly the same value when the shop is ordering goods from warehouses, otherwise, the shop may go bankrupt if its margins are to low to cover exchange rate fluctuations.
- Shorting: when cryptocurrency prices are high, it is desirable for investors to rebalance their portfolio by increasing exposure to fiat currencies or real-world commodities. However, as fiat currencies and centralized exchanges impose significant risks, it would be better to buy fiat and commodity substitues in the form of stablecoins on decentralized exchanges.
- Lending and other decentralized finance applications: stability of collateral value is critical for many applications built on top of these services.

Centralized stablecoins, such as USDT and USDC, use a trusted party to ensure that the peg is maintained. For algorithmic stablecoins, the peg can be maintained in different ways depending on the implementation, e.g. rebasement of the total supply [3], or via imitating a trusted party that holds reserves and then does market interventions for rebalancing the peg. Imititating the trusted party is usually done by allowing anyone on the blockchain to create over-collateralized financial instruments, such as collateralized debt positions [2], zero-coupon bonds (as was done in the failed Yield protocol), reserve assets [10, 11, 7], or by issuing stabilizing financial instruments in case of a depeg [4].

In this work, we present Dexy, a stablecoin protocol where an algorithmic central bank performs interventions when there is a depeg. The bank stabilizes the stablecoin value in the reference market, a liquidity pool, by injecting basecoins from its reserves when the stablecoin price is below the peg. In extreme cases, when bank reserves are depleted and the stablecoin price is still below the peg, the liquidity pool price is restored by extracting stablecoins from it, these will be injected back again at a future time, when the stablecoin price is above the peg by a certain threshold. In some aspects, Dexy could resemble Fei or Gyroscope [6].

The rest of the paper is organized as follows. In Section 2, the Dexy design in general is explained, where important trust assumptions are stated. Section 3 defines the protocol specification in detail. Section 4 we consider different theorems related to the stability and design of the protocol. Finally, in Section 7 describes Dexy implementations: the gold-pegged DexyGold stablecoin and the USD-pegged USE stablecoin.

2 Dexy Design

The Dexy protocol functions via an algorithmic central bank that holds reserves in basecoins and mints stablecoins. A corresponding liquidity pool exists where stablecoins and basecoins

can be traded. The peg price used by the bank is provided by an oracle, which is assumed to be trusted and reliable. Users can interact with both the bank and the liqudity pool, but certain rules and restrictions are applied to control the price stability. Central bank interventions are determined by the ratio of the liquidity pool price to the oracle price, $\bar{r}$, which we also call the reference reserve ratio.

2.1 The Liquidity Pool

The reference market is implemented as an automated market maker (e.g. [9]). For simplicity, a modified constant-product automated market maker (CP-AMM) is used, similar to ErgoDex [1] and UniSwap [5]. This means that the liqudity pool holds a pair of basecoins and stablecoins such that the product of their amounts remains invariant before and after a trade. Users who function as liquidity providers can provide a pair of basecoins and stablecoins to obtain a corresponding amount of LP tokens. These LP tokens can later be deposited to withdraw their initial liquidity pair deposit plus accumulated fees from trading. A fee is charged to liqudity providers to prevent excessive liquidity withdrawals.

2.2 The Algorithmic Central Bank

By depositing basecoins into the bank, users can mint stablecoins. The price provided by the bank will always equal the oracle price. A fee is charged so the bank reserves can increase faster, along with a fee for the oracle usage. Under certain conditions, the bank can intervene into the liquidity pool to control its price relative to the oracle price.

2.3 Trust Assumptions

In general, for any DeFi protocol involving user funds, it is important for trust assumptions to be explicitly stated so that users can make informed decisions to the best of their ability.

The Dexy protocol has two:

- Oracle: The protocol assumes the target price oracle is reliable. This trust assumption is unavoidable, however, it can be relaxed by using a federation of oracles that produces an average price to be delivered to the central bank, instead of relying on a single entity. In any case, it is important that oracles are rewarded from protocol usage, otherwise, they will be incentivized to adversarially attack the protocol, see Section **??** for details.

- No governance: Unlike most stablecoin protocols, we do not consider governance, as it can be used to adversarially attack the protocol. We suppose that real-world instantiations that do have it should clearly state their governance-related trust assumptions.

2.4 Actions and Dynamics

The following table summarizes the different actions in the protocol, the $\bar{r}$ range when the action happens, and the restored $\bar{r}'$ range after the action happens.

Action	Range	Restore Range
FreeMint	$1 - \epsilon_{\text{FREE}} \leq \bar{r} \leq 1 + \epsilon_{\text{FREE}}$	-
ArbitrageMint	$\bar{r} > 1 + \epsilon_{\text{FREE}}$	-
InjectBasecoins	$\bar{r} < 1 - \epsilon_{\text{FREE}}$	$1 - \epsilon_{\text{FREE}} \leq \bar{r}' \leq 1 - \epsilon_{\text{IB}}$
ExtractDexy	$\bar{r} \leq 1 - \epsilon^0_{\text{ED}} < 1 - \epsilon_{\text{FREE}}$	$1 - \epsilon_{\text{FREE}} - \epsilon^1_{\text{ED}} \leq \bar{r}' \leq 1 - \epsilon_{\text{FREE}}$
ReleaseDexy	$1 + \epsilon_{\text{RD}} < \bar{r} < 1 + \epsilon_{\text{FREE}}$	$\bar{r}' \geq 1 + \epsilon_{\text{RD}}$
StoppedWithdrawals	$\bar{r} < 1 - \epsilon_{\text{FREE}}$	-

We now give a high-level description of what happens during each action. For more technical details, refer to section 3.

2.4.1 FreeMint

- $\bar{r}$ is bounded within the range $1 - \epsilon_{\text{FREE}} \leq \bar{r} \leq 1 + \epsilon_{\text{FREE}}$.
- If bounded within the range, then minting can occur for a fixed period of time T_{FREE}.
- Users can mint from the bank up to a maximum amount determined by the amount of Dexy stablecoins in the liquidity pool at the beginning of the period.
- The bank charges a fee so the bank reserves can increase faster.
- **Consequence:** Increase bank reserves and stablecoin supply when the pool price is considered stable.

2.4.2 ArbitrageMint

- $\bar{r}$ is greater than $1 + \epsilon_{\text{FREE}}$.
- If $\bar{r}$ is above the threshold for an amount of time greater than τ_{ARB}, then minting can occur for a fixed period of time T_{ARB}.
- The liquidity pool and oracle price difference, at the begining of the period, determines the max amount of dexy stablecoins to mint δ_d.
- The bank charges a fee so the bank reserves can increase faster.
- **Consequence:** Purchase stablecoins from the bank at a discount and sell them to the liquidity pool at premium, increasing the liquidity pool stablecoin supply.

2.4.3 InjectBasecoins

- $\bar{r}$ is less than $1 - \epsilon_{\text{FREE}}$.
- The $\bar{r}$ is less than the threshold for an amount of time greater than $\tau_{\text{IB}} + T_{\text{IB}}$.
- The bank still has basecoin reserves.
- The bank will inject basecoins from its reserves into the liquidity pool until the new $\bar{r}$ is within the range $1 - \epsilon_{\text{FREE}} \leq \bar{r}' \leq 1 - \epsilon_{\text{IB}}$.
- The amount of basecoins injected is a fixed percentage α_{FREE} of the banks total basecoin reserve.
- **Consequence:** The bank's basecoin reserve decreases and the liquidity pool price recovers.

2.4.4 ExtractDexy

- $\bar{r}$ is less than $1 - \epsilon^0_{\text{ED}} < 1 - \epsilon_{\text{FREE}}$.
- $\bar{r}$ is less than the threshold for an amount of time greater than $\tau_{\text{ED}} >> \tau_{\text{IB}}$
- The bank has no more basecoin reserves.
- The bank will extract stablecoins from the liquidity pool until the new $\bar{r}$ is in the range $1 - \epsilon_{\text{FREE}} - \epsilon^1_{\text{ED}} \leq \bar{r}' \leq 1 - \epsilon_{\text{FREE}}$, so essentially the stopped-withdrawl state, and then lock them to be released at a future time.
- **Consequence:** The bank removes stablecoins from the liquidity pool and the liquidity pool price recovers.

2.4.5 ReleaseDexy

- $\bar{r}$ is within the range $1 + \epsilon_{\text{RD}} < \bar{r} < 1 + \epsilon_{\text{FREE}}$.
- $\bar{r}$ is within the range for an amount of time greater than τ_{RD}.
- The bank will release the locked stablecoins back into the liquidity pool until the new $\bar{r}$ is greater than or equal to $1 + \epsilon_{\text{RD}}$
- **Consequence:** The bank releases stablecoins back into the liquidity pool such that it is prioritized over arbitrage minting.

2.4.6 StoppedWithdrawals

- $\bar{r}$ is less than $1 - \epsilon_{\text{FREE}}$.
- Liquidity pair withdrawals are stopped without delay once the $\bar{r}$ intervention threshold is met.
- Only swaps between basecoins and stablecoins is allowed.
- **Consequence:** Liquidity providers, i.e. LP token holders, cannot withdraw their liquidity pair from the liquidity pool to prevent further price depegging.

3 Protocol Specification

We now describe the Dexy protocol specification, which includes immutable protocol paramters and state variables. We note the distinction between an action period, which measures how long before a bank intervention can happen, and a corresponding action bank delay, which measures how long before the start of the first bank action period. For a particular bank action, there may be multiple periods before a new delay is required.

The central bank parameters are a tuple C_{BANK} of:

$$\left(\phi^{\text{FREE}}_{\text{BANK}}, \phi^{\text{ARB}}_{\text{BANK}}, \epsilon_{\text{FREE}}, \epsilon_{\text{IB}}, \epsilon^{0}_{\text{ED}}, \epsilon^{1}_{\text{ED}}, \epsilon_{\text{RD}}, T_{\text{FREE}}, T_{\text{ARB}}, T_{\text{IB}}, \tau_{\text{IB}}, \tau_{\text{ED}}, \tau_{\text{RD}}, \tau_{\text{TX}}, \alpha_{\text{FREE}}, \alpha_{\text{IB}}\right)$$

- $\phi^{\text{FREE}}_{\text{BANK}} \in (0, 1)$ is the bank fee for the free-mint action.
- $\phi^{\text{ARB}}_{\text{BANK}} \in (0, 1)$ is the bank fee for the arbitrage-mint action.
- $\epsilon_{\text{FREE}} \in (0, 1)$ defines the upper and lower bounds of the free-mint range, representing the percentage that the LP price deviates from the oracle price.
- $0 < \epsilon_{\text{IB}} < \epsilon_{\text{FREE}}$ defines the percentage where the inject-basecoins action will restore the price.
- $\epsilon_{\text{FREE}} < \epsilon^{0}_{\text{ED}} < 1$ defines the percentage where the extract-dexy action will happen.
- $0 < \epsilon^{1}_{\text{ED}} < \epsilon_{\text{FREE}}$ defines the percentage where the extract-dexy action will restore the price.
- $\epsilon_{\text{RD}} < \epsilon_{\text{FREE}}$ defines the percentage where the release-dexy action will happen.
- $T_{\text{FREE}} \in \mathbb{R}_{>0}$ is the free-mint period.
- $T_{\text{ARB}} \in \mathbb{R}_{>0}$ is the arbitrage-mint period.

- $T_{\text{IB}} \in \mathbb{R}_{>0}$ is the inject-basecoins period.
- $T_{\text{BURN}} \in \mathbb{R}_{>0}$ is the extract-dexy and release-dexy period.
- $\tau_{\text{ARB}} \in \mathbb{R}_{>0}$ is the arbitrage-mint bank delay.
- $\tau_{\text{IB}} \in \mathbb{R}_{>0}$ is the inject-basecoins bank delay.
- $\tau_{\text{ED}} >> \tau_{\text{IB}}$ is the extract-dexy bank delay, which must be much larger than the inject-basecoins bank delay, since extraction is an action of last-resort.
- $\tau_{\text{RD}} \in \mathbb{R}_{>0}$ is the release-dexy bank delay.
- $\tau_{\text{TX}} \in \mathbb{R}_{>0}$ is the allowed transaction propogation delay.
- $\alpha_{\text{FREE}} \in (0, 1)$ is the percentage of dexy liquidity, in the liquidity pool, that determines the max amount of dexy that can be minted from the bank during the free-mint period.
- $\alpha_{\text{IB}} \in (0, 1)$ is the percentage of the bank's basecoin reserve that determines the max amount of basecoins to inject into the liquidity pool during an inject-basecoin action.

The central bank state is determined by a tuple of two values:

$$s_{\text{BANK}} = (R, S_D)$$

- U_{BANK} is a finite set of users that interact with the bank to mint stablecoins.
- $R \in \mathbb{R}_{>0}$ is the bank's basecoins reserves.
- $S_D \in \mathbb{R}_{>0}$ is the total supply of minted dexy stablecoins.

The liquidity pool has two parameter:

$$C_{LP} = \left(\phi_{LP}^{REDEEM}, \phi_{LP}^{SWAP}\right)$$

- U_{LP} is a finite set of users that interact with the liquidity pool, either as liquidity providers or traders.
- $\phi_{LP}^{REDEEM} \in (0, 1)$ is the liquidity provider withdrawal fee, which should be high enough to disincentivize frequent withdrawals.
- $\phi_{LP}^{SWAP} \in (0, 1)$ is the liquidity pool swap fee, which is used to reward liquidity providers.

The liquidity pool state is determined by a tuple of three values:

$$s_{\text{LP}} = (B, D, S_{LP})$$

- $B \in \mathbb{R}_{>0}$ is the amount of basecoins in the liquidity pool.
- $D \in \mathbb{R}_{>0}$ is the amount of dexy stablecoins in the liquidity pool.
- $S_{LP} \in \mathbb{R}_{>0}$ is the supply of LP tokens provided to liquidity providers.

The oracle parameters are assumed to only be the following:

$$C_O = (\phi_O)$$

- $\phi_O \in (0, 1)$ is the oracle fee.

The oracle state is determined by a tuple of two values:

$$s_{\text{O}} = (P^*_{\text{D}}, F)$$

- $P^*_{\text{D}} \in \mathbb{R}_{>0}$ is the target stablecoin price, in units of $\frac{\text{BASECOIN}}{\text{PEG ASSET}}$.
- $F \in \mathbb{R}_{\geq 0}$ is the accumulated fees of the oracle.

For the sake of clarity, we now define separate state variables for the different bank actions, but these can also be considered as part of the bank state.

The FreeMint action has the following state:

$$s_{\text{FREE}} = (t, d)$$

- $t \in \mathbb{R}_{>0}$ is the time when the free-mint period ends.
- $d \in \mathbb{R}_{\geq 0}$ is the remaining amount of dexy stablecoins that can be minted during the current free-mint period, determined by t.

The ArbMint action has the following state:

$$s_{\text{ARB}} = (t, d, \tau)$$

- $t \in \mathbb{R}_{>0}$ is the time when the arbitrage-mint period ends.
- $d \in \mathbb{R}_{\geq 0}$ is the remaining amount of dexy stablecoins that can be minted during the current arbitrage-mint period, determined by t.

- $\tau \in \mathbb{R}_{>0} \cup \{\infty\}$ is the time when $r > 1 + \epsilon_{\text{FREE}}$. This can occur more than once as time goes on, each occurrence corresponding to a potentially new arbitrage-mint period.

The InjectBasecoin action has the following state:

$$s_{\text{IB}} = (t, \tau)$$

- $t \in \mathbb{R}_{>0}$ is the time when the previous inject-basecoin action occurred.
- $\tau \in \mathbb{R}_{>0} \cup \{\infty\}$ is the time when $r < 1 + \epsilon_{\text{FREE}}$. This can occur more than once as time goes on, each occurrence corresponding to a potentially new inject-basecoin period.

The ExtractDexy and ReleaseDexy actions share the following state:

$$s_{\text{BURN}} = \left(t_{\text{ED}}, t_{\text{RD}}, d, \tau^{\text{ED}}, \tau^{\text{RD}}\right)$$

- $t_{\text{ED}} \in \mathbb{R}_{>0}$ is the time when the previous extract-dexy action occurred.
- $t_{\text{RD}} \in \mathbb{R}_{>0}$ is the time when the previous release-dexy action occurred.
- $d \in \mathbb{R}_{\geq 0}$ is the remaining dexy amount to be released.
- $\tau^{\text{ED}} \in \mathbb{R}_{>0} \cup \{\infty\}$ is the time when $r \leq 1 - \epsilon^0_{\text{ED}}$.
- $\tau^{\text{RD}} \in \mathbb{R}_{>0} \cup \{\infty\}$ is the time when $r > 1 + \epsilon_{\text{RD}}$.

3.1 Auxiliary Definitions

Definition 1 (Bank Liabilities)**.** *The bank liabilities is defined as the product of the oracle price and the minted supply of dexy stablecoins:*

$$L(P^*_{\text{D}}, s_{BANK}) = P^*_{\text{D}} S_{\text{D}} \tag{1}$$

Definition 2 (Bank Reserve Ratio)**.** *The bank reserve ratio is defined as the ratio of its reserves to its liabilities:*

$$r(P^*_{\text{D}}, s_{BANK}) = \frac{R}{L(P^*_{\text{D}}, s_{BANK})} = \frac{R}{P^*_{\text{D}} S_{\text{D}}} \tag{2}$$

Assuming the oracle price is constant, and the reserves are greater than the liabilities, the bank reserve ratio will always increase after stablecoins are minted. During periods of sudden basecoin price crashes, the reserve ratio will decrease. Since users cannot sell dexy stablecoins to the bank, the bank is liable to no one, thus the bank reserves can be less than its liabilities, which is expected during bank interventions. So, a measure of the health of the $Dexy$ protocol is having an increasing reserve ratio, before and after bank interventions.

Definition 3 (Reference Price)**.** *The liquidity pool price, or the reference market price, is defined as follows:*

$$P_{\mathrm{D}}^{\mathrm{LP}}(s_{\mathrm{LP}}) = \frac{B}{D} \tag{3}$$

The reference price, $P_{\mathrm{D}}^{\mathrm{LP}}$, has units of $\frac{\textit{BASECOIN}}{\textit{STABLECOIN}}$.

Definition 4 (Reference Reserve Ratio)**.** *The ratio of the liquidity pool price to the oracle price is called the reference reserve ratio:*

$$\bar{r}(P_{\mathrm{D}}^{*}, s_{\mathrm{LP}}) = \frac{P_{\mathrm{D}}^{\mathrm{LP}}(s_{\mathrm{LP}})}{P_{\mathrm{D}}^{*}} = \frac{B}{P_{\mathrm{D}}^{*} D} \tag{4}$$

The aim of the $Dexy$ protocol is to maintain $\bar{r}$ within a sufficiently small neighbourhood of 1. $\bar{r}$ is called the reference reserve ratio since we can interpret the formula as the reserve ratio of the liquidity pool.

Definition 5 (Arbitrage Mint Maximum)**.** *In the arbitrage mint state, when $\bar{r} > 1 + \epsilon_{\textit{FREE}}$, the maximum amount of stablecoins the bank will allow to be minted is:*

$$\delta_d = \frac{\delta_b}{\tilde{P}_{\mathrm{D}}^{*}} \tag{5}$$

Where:

$$\delta_b = B - D * \tilde{P}_{\mathrm{D}}^{*} \tag{6}$$

$$\tilde{P}_{\mathrm{D}}^{*} = P_{\mathrm{D}}^{*}(1 + \phi_{\textit{BANK}}^{\textit{FREE}} + \phi_O) \tag{7}$$

3.2 State Update Procedures

In order to perform each bank action, we must know both the current value of $\bar{r}$ but also when the transition from one region to another occurred. The following state update procedures ensure that the bank always has the accurate state information available before any bank action is performed. The procedures are meant to be continuously running, checking which bank action states can be updated according to whatever value the $\bar{r}$ has.

In practice, on eUTxO blockchains, these procedures are validators, such that if they evaluate to TRUE, then the corresponding transaction built to update the state can be accepted by a node and successfully submitted to the blockchain for inclusion.

3.2.1 UpdateArbitrageMint

Algorithm 1 UpdateArbitrageMint

Require: $s_{\text{O}}, s_{\text{LP}}, s_{\text{ARB}}$, Update time t', Current time t

$(\tilde{t}, \tilde{d}, \tilde{\tau}) \leftarrow s_{\text{ARB}}$
$(P^*_{\text{D}}, F) \leftarrow s_{\text{O}}$
$\bar{r} \leftarrow \bar{r}(P^*_{\text{D}}, s_{\text{LP}})$
$notUpdated \leftarrow (\tau == \infty)$
$validUpdate \leftarrow (t' \geq t - \tau_{\text{TX}}) \wedge (t' \leq t)$ ▷ Update time is in a valid interval.
$notReset \leftarrow (\tau < \infty)$
$validReset \leftarrow (t' == \infty)$
$isUpdate \leftarrow (\bar{r} > 1 + \epsilon_{\text{FREE}}) \wedge notUpdated \wedge validUpdate$
$isReset \leftarrow (\bar{r} \leq 1 + \epsilon_{\text{FREE}}) \wedge notReset \wedge validReset$
if $isUpdate \vee isReset$ **then**
    $s_{\text{ARB}} \leftarrow (\tilde{t}, \tilde{d}, t')$
    return TRUE
else
    return FALSE
end if

3.2.2 UpdateInjectBasecoins

Algorithm 2 UpdateInjectBasecoins

Require: $s_{\mathrm{O}}, s_{\mathrm{LP}}, s_{\mathrm{IB}}$, Update time t', Current time t

$(\tilde{t}, \tilde{\tau}) \leftarrow s_{\mathrm{IB}}$
$(P^*_{\mathrm{D}}, F) \leftarrow s_{\mathrm{O}}$
$\bar{r} \leftarrow \bar{r}(P^*_{\mathrm{D}}, s_{\mathrm{LP}})$
$notUpdated \leftarrow (\tau == \infty)$
$validUpdate \leftarrow (t' \geq t - \tau_{\mathrm{TX}}) \wedge (t' \leq t)$ ▷ Update time is in a valid interval.
$notReset \leftarrow (\tau < \infty)$
$validReset \leftarrow (t' == \infty)$
$isUpdate \leftarrow (\bar{r} < 1 - \epsilon_{\mathrm{FREE}}) \wedge notUpdated \wedge validUpdate$
$isReset \leftarrow (\bar{r} \geq 1 - \epsilon_{\mathrm{FREE}}) \wedge notReset \wedge validReset$
if $isUpdate \vee isReset$ **then**
    $s_{\mathrm{IB}} \leftarrow (\tilde{t}, t')$
    return TRUE
else
    return FALSE
end if

3.2.3 UpdateExtractDexy

Algorithm 3 UpdateExtractDexy

Require: $s_{\text{O}}, s_{\text{LP}}, s_{\text{BURN}}$, Update time t', Current time t

$(\tilde{t}_{\text{ED}}, \tilde{t}_{\text{RD}}, \tilde{d}, \tilde{\tau}^{\text{ED}}, \tilde{\tau}^{\text{RD}}) \leftarrow s_{\text{BURN}}$
$(P^*_{\text{D}}, F) \leftarrow s_{\text{O}}$
$\bar{r} \leftarrow \bar{r}(P^*_{\text{D}}, s_{\text{LP}})$
$notUpdated \leftarrow (\tau == \infty)$
$validUpdate \leftarrow (t' \geq t - \tau_{\text{TX}}) \wedge (t' \leq t)$ ▷ Update time is in a valid interval.
$notReset \leftarrow (\tau < \infty)$
$validReset \leftarrow (t' == \infty)$
$isUpdate \leftarrow (\bar{r} \leq 1 - \epsilon^0_{\text{ED}}) \wedge notUpdated \wedge validUpdate$
$isReset \leftarrow (\bar{r} > 1 - \epsilon^0_{\text{ED}}) \wedge notReset \wedge validReset$
if $isUpdate \vee isReset$ **then**
    $s_{\text{BURN}} \leftarrow (\tilde{t}_{\text{ED}}, \tilde{t}_{\text{RD}}, \tilde{d}, t', \tilde{\tau}^{\text{RD}})$
    return TRUE
else
    return FALSE
end if

3.2.4 UpdateReleaseDexy

Algorithm 4 UpdateReleaseDexy

Require: $s_{\text{O}}, s_{\text{LP}}, s_{\text{BURN}}$, Update time t', Current time t

$(\tilde{t}_{\text{ED}}, \tilde{t}_{\text{RD}}, \tilde{d}, \tilde{\tau}^{\text{ED}}, \tilde{\tau}^{\text{RD}}) \leftarrow s_{\text{BURN}}$
$(P^*_{\text{D}}, F) \leftarrow s_{\text{O}}$
$\bar{r} \leftarrow \bar{r}(P^*_{\text{D}}, s_{\text{LP}})$
$notUpdated \leftarrow (\tau == \infty)$
$validUpdate \leftarrow (t' \geq t - \tau_{\text{TX}}) \wedge (t' \leq t)$ ▷ Update time is in a valid interval.
$notReset \leftarrow (\tau < \infty)$
$validReset \leftarrow (t' == \infty)$
$isUpdate \leftarrow (\bar{r} > 1 + \epsilon_{\text{RD}}) \wedge notUpdated \wedge validUpdate$
$isReset \leftarrow (\bar{r} \leq 1 + \epsilon_{\text{RD}}) \wedge notReset \wedge validReset$
if $isUpdate \vee isReset$ **then**
    $s_{\text{BURN}} \leftarrow (\tilde{t}_{\text{ED}}, \tilde{t}_{\text{RD}}, \tilde{d}, \tilde{\tau}^{\text{ED}}, t')$
    return TRUE
else
    return FALSE
end if

3.3 Protocol Operations

3.3.1 FreeMint

Algorithm 5 FreeMint

Require: $s_{\text{O}}, s_{\text{LP}}, s_{\text{BANK}}, s_{\text{FREE}}$, Basecoins b, Updated period end time T', Current time t

$(\tilde{t}, \tilde{d}) \leftarrow s_{\text{FREE}}$
$(R, S_{\text{D}}) \leftarrow s_{\text{BANK}}$
$(B, D) \leftarrow s_{\text{LP}}$
$(P^*_{\text{D}}, F) \leftarrow s_{\text{O}}$
$\bar{r} \leftarrow \bar{r}(P^*_{\text{D}}, s_{\text{LP}})$
$\tilde{P}^*_{\text{D}} \leftarrow P^*_{\text{D}}(1 + \phi^{\text{FREE}}_{\text{BANK}} + \phi_O)$
$isCounterReset \leftarrow (t > \tilde{t})$ ▷ If true, then the protocol is in a new FreeMint period.
$dexyMinted \leftarrow \frac{b}{\tilde{P}^*_{\text{D}}}$
$oracleFeeAmount \leftarrow dexyMinted * P^*_{\text{D}} * \phi_O$
$\delta_d \leftarrow \alpha_{\text{FREE}} * D$ ▷ The maximum bank reserve growth during the FreeMint period.
$dexyAvailable \leftarrow isCounterReset \; ? \; \delta_d : \tilde{d}$
$validAmount \leftarrow dexyMinted \leq dexyAvailable$
if $\neg isCounterReset$ **then**
    $T' \leftarrow \tilde{t}$
    $validUpdate \leftarrow$ TRUE
else
    $validUpdate \leftarrow T' \in [t + T_{\text{FREE}}, t + T_{\text{FREE}} + \tau_{\text{TX}}]$ ▷ Checks new period end time.
end if
$validRatio \leftarrow \bar{r} \in (1 - \epsilon_{\text{FREE}}, 1 + \epsilon_{\text{FREE}})$
if $validUpdate \wedge validRatio$ **then**
    $s_{\text{BANK}} \leftarrow (R + b, S_{\text{D}} + dexyMinted)$
    $s_{\text{O}} \leftarrow (P^*_{\text{D}}, F + oracleFeeAmount)$
    $s_{\text{FREE}} \leftarrow (T', dexyAvailable - dexyMinted)$
    return TRUE
else
    return FALSE
end if

3.3.2 ArbitrageMint

Algorithm 6 ArbMint

Require: $s_{\text{O}}, s_{\text{LP}}, s_{\text{BANK}}, s_{\text{ARB}}$, Basecoins b, Updated period end time T', Current time t

$(\tilde{t}, \tilde{d}, \tilde{\tau}) \leftarrow s_{\text{ARB}}$
$(R, S_{\text{D}}) \leftarrow s_{\text{BANK}}$
$(B, D) \leftarrow s_{\text{LP}}$
$(P^*_{\text{D}}, F) \leftarrow s_{\text{O}}$
$\bar{r} \leftarrow \bar{r}(P^*_{\text{D}}, s_{\text{LP}})$
$\tilde{P}^*_{\text{D}} \leftarrow P^*_{\text{D}}(1 + \phi^{\text{FREE}}_{\text{BANK}} + \phi_O)$
$isCounterReset \leftarrow (t > \tilde{t})$ ▷ If true, then the protocol is in a new ArbMint period.
$dexyMinted \leftarrow \frac{b}{\tilde{P}^*_{\text{D}}}$
$oracleFeeAmount \leftarrow dexyMinted * P^*_{\text{D}} * \phi_O$
$\delta_b \leftarrow B - D * \tilde{P}^*_{\text{D}}$
$\delta_d \leftarrow \frac{\delta_b}{\tilde{P}^*_{\text{D}}}$ ▷ The maximum bank reserve growth during the ArbMint period.
$dexyAvailable \leftarrow isCounterReset \ ? \ \delta_d : \tilde{d}$
$validAmount \leftarrow dexyMinted \leq dexyAvailable$
if $\neg isCounterReset$ **then**
    $T' \leftarrow \tilde{t}$
    $validUpdate \leftarrow$ TRUE
else
    $validUpdate \leftarrow T' \in [t + T_{\text{ARB}}, t + T_{\text{ARB}} + \tau_{\text{TX}}]$ ▷ Checks new period end time.
end if
$validDelay \leftarrow t > \tilde{\tau} + \tau_{\text{ARB}}$
$validRatio \leftarrow \bar{r} > 1 + \epsilon_{\text{FREE}}$
if $validUpdate \wedge validDelay \wedge validRatio$ **then**
    $s_{\text{BANK}} \leftarrow (R + b, S_{\text{D}} + dexyMinted)$
    $s_{\text{O}} \leftarrow (P^*_{\text{D}}, F + oracleFeeAmount)$
    $s_{\text{ARB}} \leftarrow (T', dexyAvailable - dexyMinted, \tilde{\tau})$
    return TRUE
else
    return FALSE
end if

3.3.3 InjectBasecoins

Algorithm 7 InjectBasecoins

Require: $s_{\text{O}}, s_{\text{LP}}, s_{\text{BANK}}, s_{\text{IB}}$, Basecoins b, Tx creation time t', Current time t

$(\tilde{t}, \tilde{\tau}) \leftarrow s_{\text{IB}}$
$(R, S_{\text{D}}) \leftarrow s_{\text{BANK}}$
$(B, D) \leftarrow s_{\text{LP}}$
$(P^*_{\text{D}}, F) \leftarrow s_{\text{O}}$
$s'_{\text{LP}} \leftarrow (B + b, D)$
$\bar{r} \leftarrow \bar{r}(P^*_{\text{D}}, s_{\text{LP}})$
$\bar{r}' \leftarrow \bar{r}(P^*_{\text{D}}, s'_{\text{LP}})$
$validAmount \leftarrow b \leq \alpha_{\text{IB}} * R$
$validRatio \leftarrow \bar{r} < 1 - \epsilon_{\text{FREE}}$
$validInjectionRatio \leftarrow \bar{r}' \in [1 - \epsilon_{\text{FREE}}, 1 - \epsilon_{\text{IB}}]$
$validDelay \leftarrow t > \tilde{\tau} + \tau_{\text{IB}}$
$validInjectionPeriod \leftarrow t > \tilde{t} + T_{\text{IB}}$
$validUpdate \leftarrow t \leq t' + \tau_{\text{TX}}$
$validAmounts \leftarrow validAmount \wedge validRatio \wedge validInjectionRatio$
$validTimes \leftarrow validDelay \wedge validInjectionPeriod \wedge validUpdate$
if $validAmounts \wedge validTimes$ **then**
    $s_{\text{BANK}} \leftarrow (R - b, S_{\text{D}})$
    $s_{\text{LP}} \leftarrow (B + b, D)$
    $s_{\text{IB}} \leftarrow (t', \tilde{\tau})$
    return TRUE
else
    return FALSE
end if

3.3.4 ExtractDexy

Algorithm 8 ExtractDexy

Require: $s_{\text{O}}, s_{\text{LP}}, s_{\text{BURN}}$, Dexy d, Tx creation time t', Current time t

$(\tilde{t}_{\text{ED}}, \tilde{t}_{\text{RD}}, \tilde{d}, \tilde{\tau}^{\text{ED}}, \tilde{\tau}^{\text{RD}}) \leftarrow s_{\text{BURN}}$
$(B, D) \leftarrow s_{\text{LP}}$
$(P^*_{\text{D}}, F) \leftarrow s_{\text{O}}$
$s'_{\text{LP}} \leftarrow (B, D - d)$ ▷ Removing Dexy from the liquidity pool.
$\bar{r} \leftarrow \bar{r}(P^*_{\text{D}}, s_{\text{LP}})$
$\bar{r}' \leftarrow \bar{r}(P^*_{\text{D}}, s'_{\text{LP}})$
$validRatio \leftarrow \bar{r} \leq 1 - \epsilon^0_{\text{ED}} < 1 - \epsilon_{\text{FREE}}$
$validExtractionRatio \leftarrow \bar{r}' \in \left[1 - \epsilon^1_{\text{ED}}, 1 - \epsilon_{\text{FREE}}\right]$
$validDelay \leftarrow t > \tilde{\tau}^{\text{ED}} + \tau_{\text{ED}}$
$validBurnPeriod \leftarrow t > \tilde{t}_{\text{ED}} + T_{\text{BURN}}$
$validUpdate \leftarrow t \leq t' + \tau_{\text{TX}}$
$validAmounts \leftarrow validRatio \land validExtractionRatio$
$validTimes \leftarrow validDelay \land validBurnPeriod \land validUpdate$
if $validAmounts \land validTimes$ **then**
    $s_{\text{LP}} \leftarrow (B, D - d)$
    $s_{\text{BURN}} \leftarrow (t', \tilde{t}_{\text{RD}}, d, \tilde{\tau}^{\text{ED}}, \tilde{\tau}^{\text{RD}})$
    return TRUE
else
    return FALSE
end if

3.3.5 ReleaseDexy

Algorithm 9 ReleaseDexy

Require: $s_{\text{O}}, s_{\text{LP}}, s_{\text{BURN}}$, Dexy d, Tx creation time t', Current time t

$(\tilde{t}_{\text{ED}}, \tilde{t}_{\text{RD}}, \tilde{d}, \tilde{\tau}^{\text{ED}}, \tilde{\tau}^{\text{RD}}) \leftarrow s_{\text{BURN}}$
$(B, D) \leftarrow s_{\text{LP}}$
$(P^*_{\text{D}}, F) \leftarrow s_{\text{O}}$
$s'_{\text{LP}} \leftarrow (B, D + d)$ ▷ Adding Dexy back into the liquidity pool.
$\bar{r} \leftarrow \bar{r}(P^*_{\text{D}}, s_{\text{LP}})$
$\bar{r}' \leftarrow \bar{r}(P^*_{\text{D}}, s'_{\text{LP}})$
$validRatio \leftarrow \bar{r} \in (1 + \epsilon_{\text{RD}}, 1 + \epsilon_{\text{FREE}})$
$validInjectionRatio \leftarrow \bar{r}' < 1 + \epsilon_{\text{RD}}$
$validDelay \leftarrow t > \tilde{\tau}^{\text{RD}} + \tau_{\text{RD}}$
$validBurnPeriod \leftarrow t > \tilde{t}_{\text{RD}} + T_{\text{BURN}}$
$validUpdate \leftarrow t \leq t' + \tau_{\text{TX}}$
$validInjectionAmount \leftarrow d \in (0, \tilde{d})$
$validAmounts \leftarrow validRatio \land validInjectionRatio \land validInjectionAmount$
$validTimes \leftarrow validDelay \land validBurnPeriod \land validUpdate$
if $validAmounts \land validTimes$ **then**
    $s_{\text{LP}} \leftarrow (B, D + d)$
    $s_{\text{BURN}} \leftarrow (\tilde{t}_{\text{ED}}, t', \tilde{d} - d, \tilde{\tau}^{\text{ED}}, \tilde{\tau}^{\text{RD}})$
    return TRUE
else
    return FALSE
end if

4 Theorems

In all theorems, we assume that all algorithms from section 3 run correctly so that we need only worry about their return values.

Though trivial, the first stability property we consider is the tolerance of the protocol's liquidity pool to price crashes.

Theorem 1 (Market Crash Tolerance)**.** *The liqudity pool can tolerate a basecoin price crash of at most ϵ_{FREE} before any intervention.*

Proof. By construction of the protocol, no intervention will happen unless $\bar{r} < 1 - \epsilon_{\text{FREE}}$. □

Next, we consider the time it takes for the liquidity pool to recover from a price crash.

Theorem 2 (Recovery Convergence Time)**.** *Suppose there is a basecoin price crash greater than ϵ_{FREE} such that $P_{\text{D}}^{*} = s$. Assuming there is no market correction from users $u \in Y \subseteq U_{LP}$, then the liquidity pool price P_{D}^{LP} will recover in a time at most $t \leq \tau_{IB} + T_{IB}$.*

Proof. If the basecoin price crashes more than ϵ_{FREE}, then UpdateInjectBasecoins will return TRUE.

Since InjectBasecoins returns two possible values, we consider each case:

- If InjectBasecoins returns TRUE, then we know that at least $(\tau_{\text{IB}} + T_{\text{IB}})$ amount of time has elapsed.
- If InjectBasecoins returns FALSE, then, modulo the non-time based conditions that we assume are TRUE, either:
  - A time $(\tau_{\text{IB}} + T_{\text{IB}})$ has not passed yet.
  - A previous intervention happened before the current one, where $s_{\text{IB}}.\tau$ has not changed, so that we must wait at most a time T_{IB} before InjectBasecoins can be called again, since it will be the case that $(t - s_{\text{IB}}.t) < T_{\text{IB}}$, where t is the current time.

  So, for this case, we will need to wait a time within the interval $t \in [T_{\text{IB}}, \tau_{\text{IB}} + T_{\text{IB}}]$.

Thus, we will need to wait a time at most $t \leq \tau_{\text{IB}} + T_{\text{IB}}$ such that the liquidity price $P_{\text{D}}^{\text{LP}} = s$. □

When there is a crash of the basecoin price, in units of the peg asset as provided by the oracle, there will be a time delay of T_{IB} before the bank will intervene into the liquidity pool. This is done to give a chance for the liquidity pool price to stabilize itself such that the bank intervention will not be necessary.

In the worst-case scenario, the market, i.e. a subset of U_{LP}, fails to correct the liquidity pool price on its own, within the time period T_{IB}, such that $\bar{r}$ is again in the free-mint range. Only then will the bank inject part of its basecoin reserve into the liquidity pool to stabilize its price.

We have the following theorem and corollary quantifying what this minimum basecoin reserve threshold should be in the worst-case scenario.

Theorem 3 (Minimum Bank Reserve)**.** *Let s be the oracle price after a basecoin price crash such that $\bar{r} < 1 - \epsilon_{FREE}$. Assuming that all users with remaining dexy stablecoins, $u \in Y \subseteq U_{LP}$, collude such that, after a bank intervention, all stablecoins outside of the liquidity pool are sold at the new liquidity pool price s, then the bank will need a minimum basecoin reserve of $R_{min} \geq sD(\sqrt{\bar{r}} - \bar{r}) + s(S_{\text{D}} - D)$ to restore the liquidity pool price.*

Proof. Let B' and D' be the balance of basecoins and dexy, respectively, in the liqudity pool after the initial inject basecoin bank intervention. Then, by the CP-AMM formula, we have that $BD = B'D'$. The bank must inject R_1 basecoins into the liqudity pool such that the new balance will be $B' = B + R_1$. The new liquidity pool price will be $s = \frac{B'}{D'}$, so we then have that $D' = \frac{B+R_1}{s}$. We thus obtain the following relation:

$$BD = B'D' = (B + R_1)\left(\frac{B + R_1}{s}\right) = \frac{(B + R_1)^2}{s} \tag{8}$$

$$\implies R_1 = \sqrt{sBD} - B \tag{9}$$

Let D_{OUT} be the supply of dexy stablecoins outside the liqudity pool at the time of the intervention. Using the bank state s_{BANK}, and the liquidity pool state s_{LP}, we can calculate the value of this quantity such that $D_{OUT} = S_{\text{D}} - D$. Thus, to compensate for all D_{OUT} stablecoins being sold in the liquidity pool after the intervention, the bank must additionally have an amount of reserves of at least:

$$R_2 \geq sD_{OUT} = s(S_{\text{D}} - D) \tag{10}$$

So, cumulatively, we now have that:

$$R_{min} \geq R_1 + R_2 = \sqrt{sBD} - B + s\left(S_{\text{D}} - D\right) \tag{11}$$

During an intervention, $\bar{r} = \frac{B}{sD} < 1 - \epsilon_{\text{FREE}}$, by construction of the protocol. So, it follows that $B = \bar{r}sD$. We can then substitue this into equation 11 to obtain the following:

$$R_{min} \geq \sqrt{sBD} - B + s\left(S_{\mathrm{D}} - D\right) \tag{12}$$
$$= \sqrt{\bar{r}s^2D^2} - \bar{r}sD + s\left(S_{\mathrm{D}} - D\right) \tag{13}$$
$$= sD\left(\sqrt{\bar{r}} - \bar{r}\right) + s\left(S_{\mathrm{D}} - D\right) \tag{14}$$

□

Since it is expected that R_2 is very large, it would be unrealistic for the protocol to prevent bank interventions before the bank's reserves reached R_{min}. The $\bar{r}$ could fall below the free-mint range threshold much sooner than the bank could adequately fill its reserves from minting fees. Thus, the following corollary provides a practical minimum threshold for the bank's basecoin reserves, expressed as a fraction of the liquidity pool dexy stablecoin volume.

Corollary 1 (Minimum Liquidity Pool Volume Ratio)**.** *Neglecting contributions from dexy stablecoins outside the liquidity pool when $\bar{r} < 1 - \epsilon_{FREE}$, the bank's basecoin reserves should at least be $(\sqrt{\bar{r}} - \bar{r})$ of the liquidity pool's dexy stablecoin volume after the basecoin price crash.*

Proof. From Theorem 3, we have that:

$$R_{min} \geq sD\left(\sqrt{\bar{r}} - \bar{r}\right) + s\left(S_{\mathrm{D}} - D\right) \tag{15}$$
$$> sD\left(\sqrt{\bar{r}} - \bar{r}\right) \tag{16}$$
$$\tag{17}$$

Since sD has units of basecoins, this value represents the volume of dexy stablecoins in the liquidity pool. Due to the basecoin price crash, it follows that $s > P_{\mathrm{D}}^{\mathrm{LP}} \implies sD > B$. Thus, during an inject-basecoin bank intervention, the bank's basecoin reserve, as a fraction of the dexy stablecoin volume in the liquidity pool before the intervention, should be strictly greater than:

$$\text{MLPVR} \equiv \frac{R_{min}}{sD} > \sqrt{\bar{r}} - \bar{r} \tag{18}$$

□

In section 7, implementations of the Dexy protocol set $\epsilon_{\text{FREE}} = 0.02$. So, if we suppose a bank intervention occurs at the free-mint range threshold $1 - \epsilon_{\text{FREE}} = 0.98$, then an appropriate value for the MLPVR would be roughly 1% of the liquidity pool's dexy stablecoin volume. Thus, the bank intervention would inject basecoins until either $P_{\mathrm{D}}^{\mathrm{LP}} = s$ or the MLPVR amount from its reserves is used.

5 Stability

With second stabilization mechanism (burning or extraction) in place, the price in the reference market will eventually stabilize. However, for liquidity holders, burning is painful, extraction not as much, but still not desirable. Thus, the Dexy protocol is trying to avoid it (unlike other protocols, such as Gyroscope or Fei, where redemption rate fails and falls below 1 immediately when the collateralization ratio falls under 100%) by giving markets time to self-stabilize, and then doing interventions. However, this could mean slower stabilization, in comparison with other protocols, but slower stabilization helps the bank to play with possible adversaries in an environment with information assymetry, since humans always have access to more information than the algorithmic bank, and with more flexible decision making as well.

Dexy is also cautious about minting new stablecoins, which could mean slow growth of the number of stablecoins issued. This could be inconvenient, especially for big players, but the protocol is focused on stability in the first place.

Please note that the liquidity pool is disincentivizing massive bank runs due to its constant-product nature. Actually, massive bank runs are simpler for the bank to resolve, in comparison to the slow drain of the worst-case scenario.

6 Related Works

Both the Djed [11] and Gluon [7] protocols are considered as crypto-backed algorithmic stablecoins. This is due to the fact that in each one, the entire bank reserve it tokenized, using two tokens: stablecoins and reservecoins. Stablecoins represents the liabilities of the bank, while reservecoins represent the reserve surplus, i.e. equity, after the liabilities are accounted for.

With this perspective in mind, we can view the Dexy protocol as a variant of the Djed and Gluon protocols. The Dexy stablecoins minted by the bank represent its liabilities. Instead of having reservecoins used to tokenize the bank equity, which can be minted by users, we can consider the bank as having one reservecoin all to itself. This reflects the fact that users cannot redeem their stablecoins from the bank and that the bank is liable to know one, wherein reserves can be less than liabilities during bank interventions into the liquidity pool.

7 Implementations

In this section we consider two concrete implementations of the Dexyprotocol that were made for the Ergo blockchain, a gold-pegged stablecoin named DexyGoldand a USD-pegged

stablecoin named USE. The oracles used in both stablecoins are implemented using the same underlying oracle protocol.

7.1 Oracle Pools

Oracle security is the most important issue for our stablecoin protocol, since this is the only component that does not function autonomously on-chain. Thus, the protocol needs to reward oracles.

Oracle Pools 2.0 framework [8] has support for paying rewards in custom tokens. For each instance of an oracle, representing a different asset, corresponding ORTs (Oracle Reward Tokens) will be created. These ORTs will be used to reward oracles and developers, e.g. for bounties or audits.

In the dexy protocol, only the bank minting transactions charge an oracle fee in basecoins. These basecoins are sent to a BuyBack contract, which purchases the corresponding ORT available in a LP on a DEX. The purchased ORT tokens are then distributed to the oracle operators. Thus, the oracle operators receive ORT tokens that they can then redeem for the accumulated basecoin fees in the same DEX LP.

7.2 DexyGold

The first variant of the Dexy protocol is pegged to Gold and has been implement on the Ergo blockchain. The current protocol specification outline in Section 3 is modeled after this implementation.

The ORT token for the Gold oracle pool is called GORT. The pool reports the price for 1 kilogram of Gold, however, the stablecoin protocol implementation uses 1 milligram of Gold internally as the peg.

A UI for interacting with the DexyGold protocol is available at `https://cruxfinance.io/dexy/mint`, when selecting the DexyGold option.

7.3 USE

A variant of the Dexy protocol pegged to USD has also been implemented on the Ergo blockchain. It is marketed using the name USE, since there is greater adoption of a USD stablecoin expected over only a gold pegged one, and that this stablecoin is meant to be *used*.

In Section 3, the protocol specification defined α_{IB} as the percentage of the bank's bsecoins reserve that determines the max amount of basecoins to inject into the liquidity pool. USE differs from this specification by assigning α_{IB} as the percentage of the liquidity pool instead of the bank reserve. This ensures that the bank reserves are not depleted as fast, since it is expected that it's basecoin reserves exceeds the basecoin liquidity in the liquidity pool.

A UI for viewing stats of the USEprotocol is available at https://cruxfinance.io/dexy/analytics, while interacting with the algorithmic central bank can be done at https://cruxfinance.io/dexy/mint, when selecting the USE option.

7.4 Remarks

The ErgoScript smart-contracts follow a separation-of-concerns design pattern, thus each one handles its own part of the protocol instead of having one giant contract, this results in a total of 18 ErgoScript contracts required for the implementation. These 18 contracts are orchestrated together with the corresponding off-chain software for the different protocol interactions and trackers necessary to check the status of the oracle and pool peg, along with the relevant delay periods. The smart-contracts, off-chain code, simulations, and tests can be found in the following repository https://github.com/kushti/dexy-stable.

References

[1] ErgoDEX. https://www.ergodex.io.

[2] The Maker Protocol: MakerDAO's Multi-Collateral Dai (MCD) System. [Online], 2017. https://makerdao.com/en/whitepaper/.

[3] Ampleforth protocol. [Online], 2018. https://www.ampleforth.org/.

[4] XTN - Neutrino Index Token. [Online], 2023. https://github.com/waves-exchange/neutrino-contract/blob/master/docs/Neutrino%202.0.%20XTN%20Litepaper.pdf.

[5] Hayden Adams, Noah Zinsmeister, River Keefer, Dan Robinson, and Moody Salem. Uniswap v3 core, 2021. https://app.uniswap.org/whitepaper-v3.pdf.

[6] Andrew Breslin. Gyroscope: A Self-Stabilizing, "All-Weather" Reserve-Backed Stablecoin. [Online], 2022. https://consensys.io/blog/gyroscope-a-self-stabilizing-all-weather-reserve-backed-stablecoin.

[7] Bruno Woltzenlogel Paleo, Luca D'Angelo, Mohammad Shaheer, and Giselle Reis. Gluon w: A cryptocurrency stabilization protocol. Cryptology ePrint Archive, Paper 2025/1372, 2025.

[8] scalahub. Eip-0023 oracle pool 2.0. https://github.com/ergoplatform/eips/pull/41.

[9] Jiahua Xu, Krzysztof Paruch, Simon Cousaert, and Yebo Feng. Sok: Decentralized exchanges (dex) with automated market maker (amm) protocols. *ACM Computing Surveys*, 2021.

[10] Joachim Zahnentferner, Dmytro Kaidalov, Jean-Frédéric Etienne, and Javier Díaz. Djed: A formally verified crypto-backed pegged algorithmic stablecoin. Cryptology ePrint Archive, Report 2021/1069, 2021. https://eprint.iacr.org/2021/1069.

[11] Joachim Zahnentferner, Dmytro Kaidalov, Jean-Frédéric Etienne, and Javier Díaz. Djed: A formally verified crypto-backed pegged autonomous stablecoin. In *IEEE International Conference on Blockchain and Cryptocurrency*, 2023. https://icbc2023.ieee-icbc.org/program/icbc-2023-final-program.

Received \jreceived